



Technical University of Denmark

DTU Compute

Department of Applied Mathematics and Computer Science

Intermediate SQL

Joins, Constraints, Views and Authorization

Chapter 4 in the Textbook
2025

©Anne Haxthausen

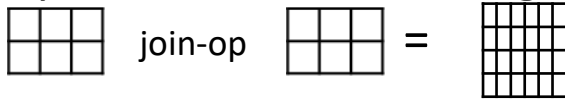
Some of the slides have been based on some slides by Flemming Schmidt. Some examples are taken from the textbook by Silberschatz, Korth, and Sudarshan.

Contents

- JOINS (section 4.1, except Note 4.1 on page 136)
- Constraints (section 4.4.1-2 & 4.4.5, and 4.5.1-3 [=4.5.1-2 in 6th edition])
- Views (section 4.2, except 4.2.3)
- Authorization (section 4.7.1-2,5-6
[= section 4.6.1-2,5-6 in (old) 6th edition])
- Demo Exercises & Exercises
- Appendix: University Database

JOIN Operations

- There are *several kinds* of join operations.
- They take two tables as arguments and return a table as result:



- The result table is a subset of the Cartesian Product of the argument tables, which
 - combines rows of the two tables, if they match some *join condition*
 - may or may not retain rows that do not match rows in the other table (by padding NULLs for missing data in the other row)
 - include more or less of the attributes
- JOIN operations are typically used in the FROM clause of a SELECT command.

Table Examples

- In the following examples two tables Courses and PreReqs have been introduced, and the following instances (being subtables of the running example instances of Course and PreReq in order to be on one page) are considered.

Courses

CourseID	Title	DeptName	Credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

PreReqs

CourseID	PreReqID
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

Cartesian Product = (INNER) JOIN

Courses

CourseID	Title	DeptName	Credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

Note: CS-315 is not listed in PreReqs!

PreReqs

CourseID	PreReqID
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

Note: CS-347 is not listed in Courses!

SELECT * FROM Courses, PreReqs;

#Cartesian Product

SELECT * FROM Courses **JOIN** PreReqs;

#Equivalent expression

CourseID	Title	DeptName	Credits	CourseID	PreReqID
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	BIO-301	BIO-101
CS-315	Robotics	Comp. Sci.	3	BIO-301	BIO-101
BIO-301	Genetics	Biology	4	CS-190	CS-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
CS-315	Robotics	Comp. Sci.	3	CS-190	CS-101
BIO-301	Genetics	Biology	4	CS-347	CS-101
CS-190	Game Design	Comp. Sci.	4	CS-347	CS-101
CS-315	Robotics	Comp. Sci.	3	CS-347	CS-101

In general:

All attributes are included.

All row combinations are included.

Attributes(R1, R2) =

Attributes(R1) union Attributes(R2)

#Rows(R1, R2) = #Rows(R1) * #Rows(R2)

NATURAL (INNER) JOIN

Courses

CourseID	Title	DeptName	Credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

PreReqs

CourseID	PreReqID
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

SELECT * FROM Courses **NATURAL JOIN** PreReqs;

CourseID	Title	DeptName	Credits	PreReqID
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

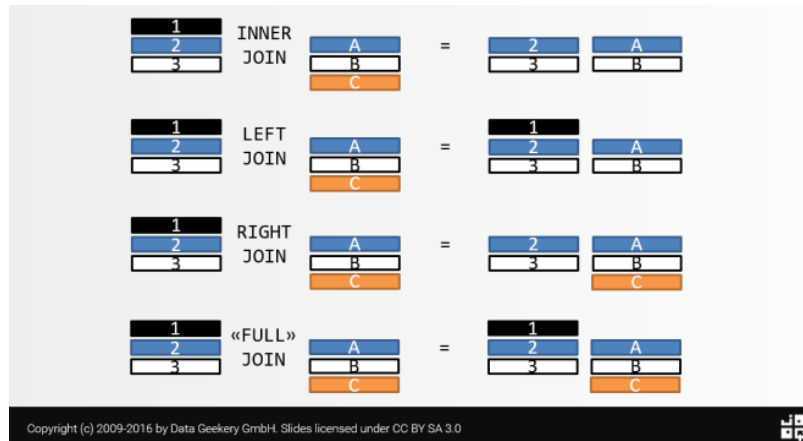
- **Included attributes:** All, but only one occurrence of common attributes ([CourseID](#)).
- **Matching:** Only rows that have the same value of the common attributes ([CourseID](#)) are joined.
- The CourseID CS-315 row in Courses is not in the result, because the JOIN cannot find a matching row in PreReqs. Same situation for CourseID CS-347 in PreReqs. So **NATURAL JOIN** *might loose important information*.

OUTER JOINS

- Three kinds of **OUTER JOINS** to avoid losing info:
 - LEFT OUTER JOIN** preserves the rows in the left table
 - RIGHT OUTER JOIN** preserves the rows in the right table
 - FULL OUTER JOIN** preserves the rows in both tables

Keyword **OUTER** can be left out without changing the meaning.

- Each outer join first computes the corresponding inner join and then adds tuples where NULL is padded for missing data:



Copyright (c) 2009-2016 by Data Geekery GmbH. Slides licensed under CC BY SA 3.0

NATURAL LEFT OUTER JOIN

Courses

CourseID	Title	DeptName	Credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

PreReqs

CourseID	PreReqID
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

SELECT * FROM Courses **NATURAL LEFT OUTER JOIN** PreReqs;

CourseID	Title	DeptName	Credits	PreReqID
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	NULL

- Note: The result contains the left table with extra right table attributes.

NATURAL RIGHT OUTER JOIN

Courses

CourseID	Title	DeptName	Credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

PreReqs

CourseID	PreReqID
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

SELECT * FROM Courses **NATURAL RIGHT OUTER JOIN** PreReqs;

CourseID	PreReqID	Title	DeptName	Credits
BIO-301	BIO-101	Genetics	Biology	4
CS-190	CS-101	Game Design	Comp. Sci.	4
CS-347	CS-101	NULL	NULL	NULL

- Note: The result contains the right table with extra left table attributes
- Same as **SELECT * FROM** Prereqs **NATURAL LEFT OUTER JOIN** Courses;

NATURAL FULL OUTER JOIN

Courses

CourseID	Title	DeptName	Credits
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

PreReqs

CourseID	PreReqID
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

SELECT * FROM Courses **NATURAL FULL OUTER JOIN** PreReqs;

CourseID	Title	DeptName	Credits	PreReqID
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	NULL
CS-347	NULL	NULL	NULL	CS-101

- **NATURAL FULL OUTER JOIN** *is not supported by MariaDB and MySQL,* instead use:

```
(SELECT * FROM Courses NATURAL LEFT OUTER JOIN PreReqs)
UNION
(SELECT CourseID, Title, DeptName, Credits, PreReqID
FROM Courses NATURAL RIGHT OUTER JOIN PreReqs);
```

JOIN Conditions

- **JOIN conditions** define which rows in the two argument tables $T1$ and $T2$ match, and which attributes are present in the result of the JOIN:
 - $T1$ **NATURAL** join-op $T2$:
 - two rows match, if $T1.A = T2.A$ for each common attribute name A
 - common attribute names only occur ones in the result
 - $T1$ join-op $T2$ **USING** ($A_1, \dots, A_n$), where $A_1, \dots, A_n$ are (a subset of) common attributes:
 - two rows match, if $T1.A_1 = T2.A_1$ and ... and $T1.A_n = T2.A_n$
 - $A_1, \dots, A_n$ only occur ones in the result
 - compared to NATURAL, it allows to *not join on all* common attributes
 - $T1$ join-op $T2$ **ON predicate**:
 - two rows match, if *predicate* is true
 - all attributes are included in the result
 - allows e.g. for 'joining' attributes named differently in the two tables

where join-op can be **INNER JOIN**, **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, **FULL OUTER JOIN**,

where **INNER** and **OUTER** can be left out (without changing the meaning).

- **Note:** $T1$ join-op $T2$ **USING** ($A_1, \dots, A_n$) = $T1$ **NATURAL** join-op $T2$,
if $A_1, \dots, A_n$ include *all* common attributes.
- **Note:** $T1$ **JOIN** $T2$ is short for $T1$ **JOIN** $T2$ **ON TRUE**

Examples of USING (A₁, A₂, ..., A_n)

SELECT * FROM Courses JOIN PreReqs USING (CourseID);

SELECT * FROM Courses NATURAL JOIN PreReqs; # equivalent expression

CourseID	Title	DeptName	Credits	PreReqID
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

SELECT * FROM Courses RIGHT OUTER JOIN PreReqs USING (CourseID);

SELECT * FROM Courses NATURAL RIGHT OUTER JOIN PreReqs; # equivalent expression

CourseID	PreReqID	Title	DeptName	Credits
BIO-301	BIO-101	Genetics	Biology	4
CS-190	CS-101	Game Design	Comp. Sci.	4
CS-347	CS-101	NULL	NULL	NULL

Examples of JOIN ON Predicate

SELECT * FROM Courses **JOIN** PreReqs **ON** Courses.CourseID = PreReqs.CourseID;

SELECT * FROM Courses **JOIN** PreReqs **WHERE** Courses.CourseID = PreReqs.CourseID;

CourseID	Title	DeptName	Credits	CourseID	PreReqID
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101

SELECT * FROM Courses **LEFT OUTER JOIN** PreReqs **ON** Courses.CourseID = PreReqs.CourseID;

CourseID	Title	DeptName	Credits	CourseID	PreReqID
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101
CS-315	Robotics	Comp. Sci.	3	NULL	NULL

SELECT * FROM Courses **LEFT OUTER JOIN** PreReqs **ON TRUE**

WHERE Courses.CourseID = PreReqs.CourseID;

CourseID	Title	DeptName	Credits	CourseID	PreReqID
BIO-301	Genetics	Biology	4	BIO-301	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-190	CS-101

IMPORTANT: **ON** is a part of the join operation, while **WHERE** is calculated afterwards!

JOIN Summary

■ Syntax:

- $T1$ **JOIN** $T2$ (equivalent to: $T1 \text{ , } T2$)
- $T1$ **NATURAL** join-op $T2$
- $T1$ join-op $T2$ **USING** ($A_1, \dots, A_n$), where $A_1, \dots, A_n$ are (a subset of) common attributes
- $T1$ join-op $T2$ **ON** *predicate*

■ JOIN conditions define which rows in the two tables match, and which attributes are present in the result of the JOIN:

- **NATURAL:**
 - two rows match, if $T1.A = T2.A$ for each common attribute name A
 - common attribute names only occur ones in the result
- **USING** ($A_1, \dots, A_n$):
 - two rows match, if $T1.A_1 = T2.A_1$ and ... and $T1.A_n = T2.A_n$
 - $A_1, \dots, A_n$ only occur ones in the result
- **ON** *predicate*:
 - two rows match, if *predicate* is true
 - all attributes are included in the result

■ join-op defines how rows in each table that do not match any row in the other table (based on the JOIN condition) are treated:

- **INNER JOIN:** unmatched rows are not included
- **LEFT OUTER JOIN:** unmatched rows in the left table are included, with NULLs for missing right table values.
- **RIGHT OUTER JOIN:** unmatched rows in the right table are included, with NULLs for missing left table values.
- **FULL OUTER JOIN:** as both for **LEFT** and for **RIGHT**

Constraints

- **Integrity constraints**
 - Express **rules about allowed data** in database tables.
 - Can be **specified** in CREATE TABLE (or ALTER TABLE ADD) statements.
 - If a data changing operation would lead to a **violation** of a specified constraint, the DBMS will reject it (if the feature is implemented by the DBMS).
 - Purpose: to guard against wrong and inconsistent data in a database.
- **Integrity constraints on a single table**
 - **Data types**
 - **NOT NULL**
 - **PRIMARY KEY**
 - ...
- **Integrity constraints between two tables**
 - **FOREIGN KEY** Referential Integrity

Further checks can be made by programming **Triggers**, that fire (i.e. executes), when the DBMS gets an INSERT, UPDATE or DELETE command.

Data Types as Attribute Value Constraints

- The data type given to an attribute represents a constraint:
 - the attribute must only be assigned values of the given type.
- Data Types in MariaDB¹:
 - Numeric data types, e.g. **INT**, **DECIMAL(i,j)**, ...
 - String data types, e.g. **VARCHAR(n)**, **ENUM(s₁, ..., s_n)**
 - Date and Time datatypes, e.g. **DATE**, **TIME**, **YEAR**
 - ...

For more info, see the MariaDB documentation here:

<https://mariadb.com/kb/en/library/documentation/>

1: The data types depend on the DBMS implementation.

Example of Attribute Value Constraint

- Ensure the Semester attribute values are checked

```
CREATE TABLE Section (  
    CourseID          VARCHAR(8),  
    SectionID         VARCHAR(8),  
    Semester          ENUM('Fall', 'Winter', 'Spring', 'Summer'),  
    StudyYear         DECIMAL(4,0),  
    Building          VARCHAR(15),  
    Room              VARCHAR(7),  
    TimeSlotID        VARCHAR(4),  
    PRIMARY KEY(CourseID, SectionID, Semester, StudyYear) );  
  
INSERT Section VALUES('BIO-101','1','Summer','2009','Painter','514','B'); -- is ok  
INSERT Section VALUES('BIO-101','1','Sommer','2009','Painter','514','B'); -- is not ok
```

NOT NULL and PRIMARY KEY Constraints

■ NOT NULL

- In CREATE TABLE declare LastName and Budget to be NOT NULL:

LastName VARCHAR(20) **NOT NULL**;

Budget DECIMAL(12,2) **NOT NULL**;

- Insertion of a new row demands that LastName and Budget are given valid attribute values from the data type specified. NULL values are not allowed.

■ PRIMARY KEY($A_1, A_2, \dots, A_n$)

- For each row the PRIMARY KEY will be unique: no two rows have the same values for the primary key.
- Attributes of primary keys are automatically required to be NOT NULL.

Foreign Key Referential Integrity Constraints

Let R be a table, and K a subset of its attributes.

- K can be specified to be a **foreign key** referencing another table R' , if K is a primary key of R' .
- This implies a **referential integrity constraint on the allowed instances r of R and r' of R'** : for any row in r , there must exist a row in r' having the same values for K , unless an attribute of K is NULL.

Example:

- Attribute **DeptName** in **Instructor** is declared to be a **foreign key** referencing the **Department** table:

```
CREATE TABLE Instructor
(InstID          VARCHAR(5),
 InstName        VARCHAR(20) NOT NULL,
 DeptName        VARCHAR(20),
 Salary          DECIMAL(8,2),
 PRIMARY KEY (InstID),
 FOREIGN KEY(DeptName) REFERENCES
 Department(DeptName));
```

- This implies a **referential integrity constraint on the allowed instances of Instructor and Department**: for any row in **Instructor**, **DeptName** must be NULL, or there must exist a row in **Department** having the same values for **DeptName**.

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Department

DeptName	Building	Budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Foreign Key Referential Integrity Violations

- *SQL rejects operations* that would lead to a **violation** of the specified referential integrity constraints e.g.:

- insertions to or updates in the referencing table R (Instructor), e.g.

INSERT INTO Instructor **VALUES**

(99999, 'Anne', '**Mathematics**', 90000.00);

- deletion or updates of rows in the referenced table R' (Department), (unless referential actions are specified, see next page), e.g.

DELETE FROM Department

WHERE DeptName = 'Physics';

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Department

DeptName	Building	Budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Foreign Keys: Specification of Referential Actions

In the **CREATE TABLE** command for **R** one can write

- **ON DELETE** *referential-actions*
- **ON UPDATE** *referential-actions*

where *referential-actions* specify modifications to make in **R**, if a DELETE/UPDATE in **R'** leads to a violation, where rows in **R** have **K** values not existing in **R'**:

- **ON DELETE SET NULL** : set the **K** attributes that do not anymore exist in **R'** to **NULL** in **R**.
- **ON DELETE CASCADE** : delete the problematic rows of **R**.
- **ON UPDATE CASCADE**: make the same updates of **K** values in **R**.

■ Example:

```
CREATE TABLE Instructor
(InstID      VARCHAR(5),
 InstName    VARCHAR(20) NOT NULL,
 DeptName    VARCHAR(20),
 Salary      DECIMAL(8,2),
PRIMARY KEY (InstID),
FOREIGN KEY(DeptName) REFERENCES Department(DeptName)
ON DELETE SET NULL
ON UPDATE CASCADE);
```

Referential Actions - Examples

■ Example:

```
CREATE TABLE Instructor
(InstID          VARCHAR(5),
 InstName        VARCHAR(20) NOT NULL,
 DeptName        VARCHAR(20),
 Salary          DECIMAL(8,2),
 PRIMARY KEY (InstID),
 FOREIGN KEY(DeptName)
 REFERENCES Department(DeptName)
 ON DELETE SET NULL
 ON UPDATE CASCADE);
```

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics NULL	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics NULL	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology BioSc	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

■ Effect of ON DELETE SET NULL on

```
DELETE FROM Department
WHERE DeptName = 'Physics';
```

■ Effect of ON UPDATE CASCADE on

```
UPDATE Department SET DeptName = 'BioSc'
WHERE DeptName = 'Biology';
```

Department

DeptName	Building	Budget
Biology BioSc	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Referential Actions - Examples

■ Example:

```
CREATE TABLE Instructor
(InstID          VARCHAR(5),
 InstName        VARCHAR(20) NOT NULL,
 DeptName        VARCHAR(20),
 Salary          DECIMAL(8,2),
 PRIMARY KEY (InstID),
 FOREIGN KEY(DeptName)
    REFERENCES Department(DeptName)
    ON DELETE CASCADE);
```

■ Effect of **ON DELETE CASCADE** on

```
DELETE FROM Department
WHERE DeptName = 'Physics';
```

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Department

DeptName	Building	Budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Views

- Restricting information access
 - In some cases, it is not desirable for all users to be able see all the rows and all columns stored in a database table!
- Views
 - Provide a *mechanism to hide certain data* from the view of certain users. Specific columns can be hidden as well as specific rows.
 - A view is a *virtual table*.
 - A view *can be used as a table*, but it is a stored SQL expression, that is executed each time it is used!

Creating and Using Views

- Command to define a view with name *id*:

```
CREATE VIEW id AS query-expression
```

- *id* can be used in queries as if it had been a table name
- *id* is a short for writing *query-expression*
- A view definition is not the same as creating a new relation by evaluating the query expression,
 - rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

VIEW Example that Restricts Access

CREATE VIEW Faculty AS

SELECT InstID, InstName, DeptName **FROM** Instructor

WHERE DeptName **NOT IN** ('Finance', 'Music');

SELECT * FROM Instructor;

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

SELECT * FROM Faculty;

InstID	InstName	DeptName
10101	Srinivasan	Comp. Sci.
22222	Einstein	Physics
32343	El Said	History
33456	Gold	Physics
45565	Katz	Comp. Sci.
58583	Califieri	History
76766	Crick	Biology
83821	Brandt	Comp. Sci.
98345	Kim	Elec. Eng.

The above SQL expression is executed as:

SELECT * FROM

SELECT InstID, InstName, DeptName **FROM** Instructor

WHERE DeptName **NOT IN** ('Finance', 'Music');

- Note: In the Faculty View the Salary attribute is excluded as well as the rows with instructors working in Finance and Music.

VIEW Example with Calculations

```
CREATE VIEW DepartmentSalary AS  
SELECT DeptName, SUM(Salary) AS DeptSalary  
FROM Instructor GROUP BY DeptName;
```

```
SELECT * FROM DepartmentSalary;
```

DeptName	DeptSalary
Biology	72000.00
Comp. Sci.	232000.00
Elec. Eng.	80000.00
Finance	170000.00
History	122000.00
Music	40000.00
Physics	182000.00

- Note: Users can see only the sum of salaries per department, and not the salaries of individual instructors (i.e. assuming no user access to Instructor).

VIEW Example with Joined Tables

```
CREATE VIEW Studentview AS
SELECT StudID, StudName, TIMESTAMPDIFF(YEAR, Birth, CURRENT_DATE()) AS
Age, TotCredits, InstID
FROM Student NATURAL LEFT OUTER JOIN Advisor;
```

SELECT * FROM Studentview;

StudID	StudName	Age	TotCredits	InstID
00128	Zhang	23	102	45565
12345	Shankar	19	32	10101
19991	Brandt	22	80	NULL
23121	Chavez	23	110	76543
44553	Peltier	19	56	22222
45678	Levy	19	46	22222
54321	Williams	20	54	NULL
55739	Sanchez	20	38	NULL
70557	Snow	19	0	NULL
76543	Brown	21	58	45565
76653	Aoi	21	60	98345
98765	Bourikas	22	98	98345
98988	Tanaka	23	120	76766

SELECT MIN(Age), MAX(Age), AVG(Age)
FROM Studentview;

MIN(Age)	MAX(Age)	AVG(Age)
19	23	20.8462

- Note: Age can change every day by someone having a birthday, and the users would prefer to have the advisor InstID readily available. Views used as Tables!

VIEW Updates

■ Add a new row to the Faculty view

INSERT Faculty **VALUES** ('30765', 'Green', 'Physics');

- This insertion will be executed by the DBMS as in the statement:

INSERT Instructor **VALUES** ('30765', 'Green', 'Physics', NULL);

- Some view updates can be ambiguous and some can be impossible.

■ Most SQL systems only allow updates of simple views satisfying conditions like:

- Any attribute not listed in the view's SELECT clause can be set to NULL.
- The FROM clause has only one table, no JOIN operations!
- The SELECT clause contains only attribute names of the base table, and does not have any expressions, aggregates or DISTINCT specifications.
- The query does not have a GROUP BY or HAVING clause.

Authorization

- Concepts used to control the access to database objects:
 - users
 - roles
 - privileges

Creating and Dropping Users

■ CREATE USER with a password

- As Database Administrator (i.e. when connected/logged in as root) you can create a user *userid* with a password *password* by issuing the command:

CREATE USER *userid* **IDENTIFIED BY** *password*;

- Creation of users Thomas, Peter and Bill:

CREATE USER 'Thomas'@'localhost' **IDENTIFIED BY** '1984';

CREATE USER 'Peter'@'localhost' **IDENTIFIED BY** '1979';

CREATE USER 'Bill'@'localhost' **IDENTIFIED BY** '1980';

- Thomas, Peter and Bill can now connect/login to the database server, but they cannot access any databases or tables before privileges are granted

SHOW GRANTS FOR 'Thomas'@'localhost';

```
Grants for Thomas@localhost
GRANT USAGE ON *.* TO 'Thomas'@'localhost'
```

■ RENAME USER

- RENAME USER** 'Bill'@'localhost' **TO** 'William'@'localhost';

■ DROP USER

- DROP USER** 'William'@'localhost';

■ SHOW created users (in MySQL and MariaDB):

- SELECT** user **FROM** mysql.user;

user
root
Thomas
Peter

Changing Password

- If DBA is connected as root:

SET PASSWORD FOR 'Thomas'@'localhost' = Password ('Sec4525');

- If connected as Thomas:

SET PASSWORD = Password ('Sec4525');

GRANT and REVOKE Privileges

Syntax

The syntax for granting privileges on a table is:

```
GRANT privileges ON object TO user;
```

```
REVOKE privileges ON object FROM user;
```

privileges

It can be any of the following values:

Privilege	Description
SELECT	Ability to perform SELECT statements on the table.
INSERT	Ability to perform INSERT statements on the table.
UPDATE	Ability to perform UPDATE statements on the table.
DELETE	Ability to perform DELETE statements on the table.
INDEX	Ability to create an index on an existing table.
CREATE	Ability to perform CREATE TABLE statements.
ALTER	Ability to perform ALTER TABLE statements to change the table definition.
DROP	Ability to perform DROP TABLE statements.
GRANT OPTION	Allows you to grant the privileges that you possess to other users.
ALL	Grants all permissions except GRANT OPTION.

GRANT Privileges - Examples

- When users are created they can be granted privileges

GRANT SELECT (InstID, InstName, DeptName) **ON** University.Instructor **TO** 'Thomas'@'localhost';

Note: Thomas can only read from the named database, table and attributes!

GRANT SELECT ON University.* **TO** 'Thomas'@'localhost';

Note: Thomas can read all tables and attributes from the University Database!

GRANT SELECT ON *.* TO 'Thomas'@'localhost';

Note: Thomas can read all databases, tables and attributes on the localhost!

GRANT ALL ON University.* **TO** 'Thomas'@'localhost';
SHOW GRANTS FOR 'Thomas'@'localhost';

Note: Thomas can do whatever he likes to the University database, since he has all privileges!

Grants for Thomas@localhost
GRANT USAGE ON *.* TO 'Thomas'@'localhost'
GRANT ALL PRIVILEGES ON `university`.* TO 'Thomas'@'localhost'

The creator of an object holds all privileges on that object, incl. the privileges to grant privileges to other users.

If **WITH GRANT OPTION** is appended at the end of a **GRANT** command, the user will be allowed to grant the same privileges to other users.

Roles

- One can create *roles*, e.g:
 - **CREATE ROLE** TeachingAssistant;
- Roles can (just like users) be granted privileges, e.g:
 - **GRANT SELECT ON** University.Takes **TO** TeachingAssistant;
- Roles can be granted to users (as well as to other roles), e.g:
 - **GRANT** TeachingAssistant **TO** 'Thomas'@'localhost';
by this Thomas is granted the privileges granted to TeachingAssistant
- Roles can be dropped
 - **DROP ROLE** TeachingAssistant;
- NOTE: these commands are not supported by all DBMS (e.g. not for some MySQL versions), but are supported by MariaDB.



Summary

- Joins based on Cartesian product variations.
- Integrity Constraints to ensure a consistent data model.
- Views of tables to hide and protect data.
- Authorization by creating users & roles and granting & revoking privileges.

Readings

- In Database Systems Concepts please read sections stated on slide 2.
- Pay special attention to the Summaries

Demo Exercises



Demo Exercises clarify ideas and concepts from the Lecture to provide you with good Database Skills.

Do the Demo Exercises.
Solutions can be found on later slides.

JOINS

4.1.1 NATURAL JOIN

Use the University Database and display a list of all students with ID, name, department and building. The list should be ordered by the students IDs.

Note:

Student(StudID, StudName, Birth, DeptName, TotCredits)

Department(DeptName, Building, Budget)

JOINS

4.1.2 NATURAL LEFT OUTER JOIN

Display a list of all instructors, showing their ID, name and the number of sections that they have taught. Argue for the choice of join type.

Note:

Instructor(InstID, InstName, DeptName, Salary)
Teaches(InstID, CourseID, SectionID, Semester,
StudyYear)

4.1.3 Nested SELECT alternative

Write the same query as in the Exercise above, but use a scalar subquery (i.e. a SELECT statement that returns a single value for a number of rows) within another SELECT statement.

VIEWS

4.1.4 Create a VIEW

Called

SeniorStudents(StudID, StudName)
of students with a TotCredits > 100.

Note:

Student(StudID, StudName, Birth, DeptName,
TotCredits)

4.1.5 Create a VIEW

Called

Creditview(StudyYear, SumCredits)
giving the total number of credits taken by
students in each year.

Note:

Takes(StudID, CourseID, SectionID, Semester,
StudyYear, Grade)
Course(CourseID, Title, DeptName, Credits)

Solutions to Demo Exercises



JOINS

4.1.1 NATURAL JOIN

Use the University Database and display a list of all students with ID, name, department and building. The list should be ordered by the students IDs.

Note:

Student(StudID, StudName, Birth, DeptName, TotCredits)

Department(DeptName, Building, Budget)

4.1.1 NATURAL JOIN

SELECT StudID, StudName, DeptName, Building
FROM Student **NATURAL JOIN** Department
ORDER BY StudID;

StudID	StudName	DeptName	Building
00128	Zhang	Comp. Sci.	Taylor
12345	Shankar	Comp. Sci.	Taylor
19991	Brandt	History	Painter
23121	Chavez	Finance	Painter
44553	Peltier	Physics	Watson
45678	Levy	Physics	Watson
54321	Williams	Comp. Sci.	Taylor
55739	Sanchez	Music	Packard
70557	Snow	Physics	Watson
76543	Brown	Comp. Sci.	Taylor
76653	Aoi	Elec. Eng.	Taylor
98765	Bourikas	Elec. Eng.	Taylor
98988	Tanaka	Biology	Watson

JOINS

4.1.2 NATURAL LEFT OUTER JOIN

Display a list of all instructors, showing their ID, name and the number of sections that they have taught. Argue for the choice of join type.

Note:

Instructor(InstID, InstName, DeptName, Salary)
Teaches(InstID, CourseID, SectionID, Semester,
StudyYear)

4.1.3 Nested SELECT alternative

Write the same query as in the Exercise above, but use a scalar subquery (i.e. a SELECT statement that returns a single value for a number of rows) within another SELECT statement.

4.1.2 NATURAL LEFT OUTER JOIN

SELECT InstID, InstName, **COUNT**(SectionID) **AS** SectionsTought
FROM Instructor **NATURAL LEFT OUTER JOIN** Teaches
GROUP BY InstID, InstName;

InstID	InstName	SectionsTought
10101	Srinivasan	3
12121	Wu	1
15151	Mozart	1
22222	Einstein	1
32343	El Said	1
33456	Gold	0
45565	Katz	2
58583	Califieri	0
76543	Singh	0
76766	Crick	2
83821	Brandt	3
98345	Kim	1

LEFT OUTER is needed, otherwise instructors who do not teach would not appear in the result (with a 0).

4.1.3 Nested SELECT alternative

SELECT InstID, InstName,
(**SELECT COUNT**(*) **FROM** Teaches **WHERE** Teaches.InstID
= Instructor.InstID) **AS** SectionsTought **FROM** Instructor;

VIEWS

4.1.4 Create a VIEW

Called

SeniorStudents(StudID, StudName)
of students with a TotCredits > 100.

Note:

Student(StudID, StudName, Birth, DeptName,
TotCredits)

4.1.5 Create a VIEW

Called

Creditview(StudyYear, SumCredits)
giving the total number of credits taken by
students in each year.

Note:

Takes(StudID, CourseID, SectionID, Semester,
StudyYear, Grade)
Course(CourseID, Title, DeptName, Credits)

4.1.4 Create a VIEW

CREATE VIEW SeniorStudents **AS**
SELECT StudID, StudName **FROM** Student
WHERE TotCredits > 100;

SELECT * FROM SeniorStudents;

StudID	StudName
00128	Zhang
23121	Chavez
98988	Tanaka

4.1.5 Create a VIEW

CREATE VIEW Creditview(StudyYear, SumCredits) **AS**
(**SELECT** StudyYear, **SUM**(Credits) **FROM** Takes
NATURAL JOIN Course **GROUP BY** StudyYear);

SELECT * FROM Creditview;

StudyYear	SumCredits
2009	49
2010	29

Exercises

Please answer all exercises
to demonstrate your
Database Skills.

MySQL Exercises

Solutions are available at 11:45



Join, Constraints, View, and Authorization

4.2.1 JOINS

Display the list of all departments, with the total number of instructors in each department.

Note:

Department(DeptName, Building, Budget)
Instructor(InstID, InstName, DeptName, Salary)

4.2.2 JOINS

List titles of those courses that are prereqs of other courses. Make the solution without using WHERE.

4.2.3 JOINS

List for each course its id and title and the number of courses that is a prereq for.

4.2.4 JOINS

Display the names of active students, along with titles of the courses they take. The list should be sorted by the student names. (A student is active means: is taking a course.)

4.2.5 Referential Actions

Consider the CREATE TABLE commands for the university database in the appendix. Discuss why the referential ON DELETE actions have been chosen as they are.

Consider the database instance in the appendix. Which table(s) are changed by the following command:

DELETE FROM COURSE **WHERE** CourseID = 'BIO-301';

Hint: First answer this without using MySQL.

Afterwards, you can check your answer by executing the command.

4.2.6 CREATE a View

Called

SeniorInstructors(InstID, InstName, DeptName)
of instructors with a salary > 80000

Note:

Instructor(InstID, InstName, DeptName, Salary)

4.2.7 Authorization

a) Connect to the database as Database Administrator (root) and create the users **Karen**, **Linda** and **Susan** with the generic password **SetPassword**. Please observe that user names are case sensitive!

b) Then grant SELECT to **Karen** and ALL to **Linda** and **Susan** to a database under your DBA control.

c) Then close your connection to the server (on Workbench under Windows this can be done choosing File->Close Connection Tab).

d) Then add a connection for **Karen**. From the welcome page of Workbench this is done as follows:

1. Click on the + icon and a window will pop up.
2. In that window choose a name (e.g. **KarenConnection**) for the connection, set the username to **Karen**, check port is 3306, and click **Ok**. Then you will be returned to the welcome page where you can see the new connection **KarenConnection**.

e) Connect as **Karen**:

1. Click on the icon with the new connection name (**KarenConnection**) and a window will pop up.
2. Fill out the password choosen for **Karen** in step a): **SetPassword**. It will then probably give a connection warning and you should just click on **Continue Anyway**.

f) Change Karen's password to **KarenSecret**

g) Try (still as **Karen**) to execute some select statements and some table modifications and see what happens.

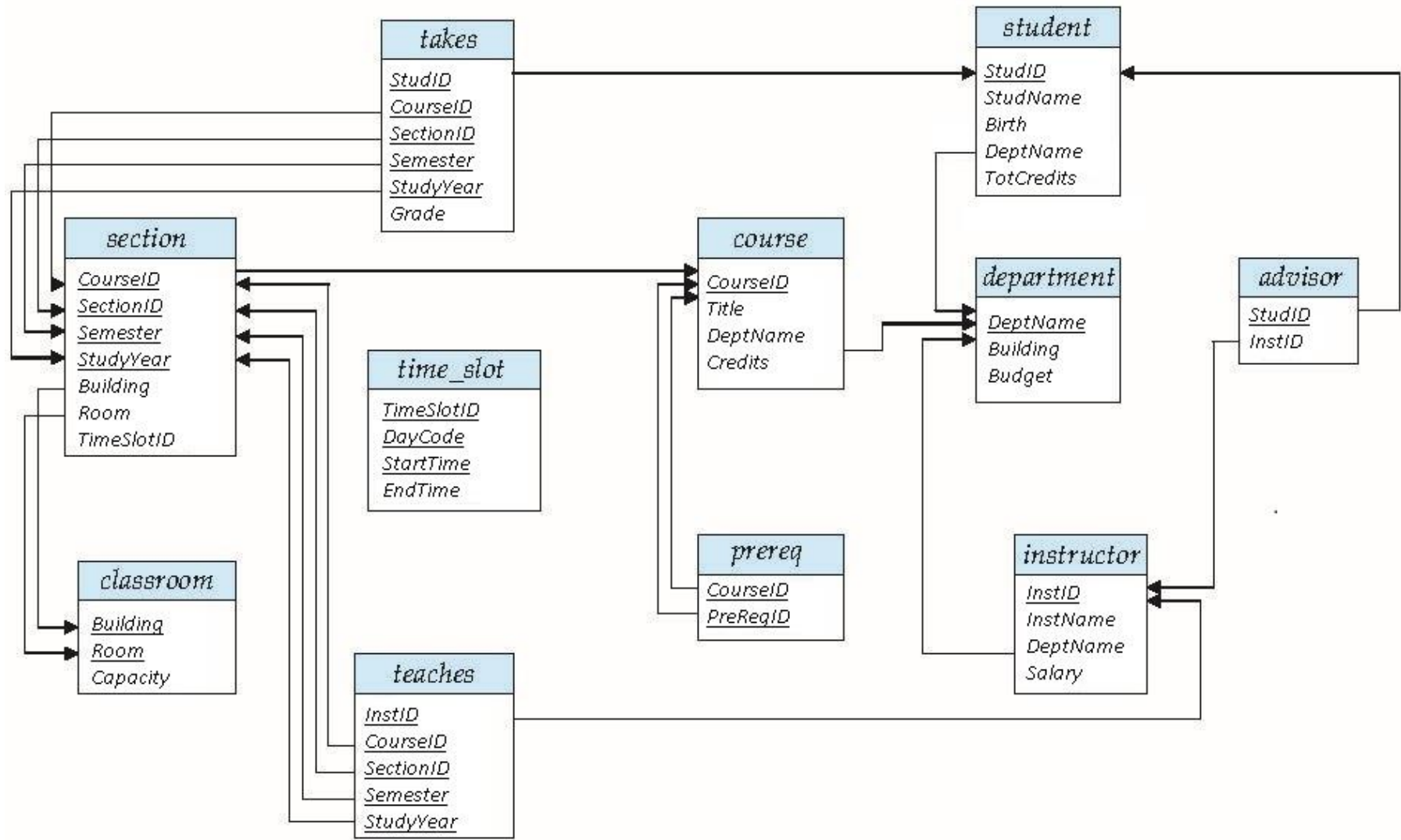
h) Close the connection and then connect as DBA (root) and drop users **Karen**, **Linda** and **Susan**.

i) Close the connection. Delete **KarenConnection** by clicking on the tools icon on the Workbench welcome page, then clicking on **KarenConnection**, and finally clicking on **delete** and then **close**.

University Database, revisited

- Database example used throughout the textbook and in this course!
- 1. Database Schema Diagram
- 2. Database Schema (CREATE TABLE Commands)
- 3. Database Instance

Database Schema Diagram from the book, but with new names



University Database Schema (1 of 4)

```
CREATE TABLE Instructor
    (InstID                VARCHAR(5),
     InstName              VARCHAR(20) NOT NULL,
     DeptName              VARCHAR(20),
     Salary                DECIMAL(8,2),
     PRIMARY KEY (InstID),
     FOREIGN KEY(DeptName) REFERENCES Department(DeptName) ON DELETE SET NULL
    );

CREATE TABLE Student
    (StudID                VARCHAR(5),
     StudName              VARCHAR(20) NOT NULL,
     Birth                 DATE,
     DeptName              VARCHAR(20),
     TotCredits            DECIMAL(3,0),
     PRIMARY KEY (StudID),
     FOREIGN KEY(DeptName) REFERENCES Department(DeptName) ON DELETE SET NULL
    );

CREATE TABLE Advisor
    (StudID                VARCHAR(5),
     InstID                VARCHAR(5),
     PRIMARY KEY (StudID),
     FOREIGN KEY(InstID) REFERENCES Instructor(InstID) ON DELETE SET NULL,
     FOREIGN KEY(StudID) REFERENCES Student(StudID) ON DELETE CASCADE
    );

CREATE TABLE Department
    (DeptName              VARCHAR(20),
     Building              VARCHAR(15),
     Budget                DECIMAL(12,2),
     PRIMARY KEY (DeptName)
    );
```

University Database Schema (2 of 4)

```
CREATE TABLE Course
    (CourseID          VARCHAR(8),
     Title             VARCHAR(50),
     DeptName          VARCHAR(20),
     Credits            DECIMAL(2,0),
     PRIMARY KEY (CourseID),
     FOREIGN KEY (DeptName) REFERENCES Department (DeptName) ON DELETE SET NULL
    );
```

```
CREATE TABLE PreReq
    (CourseID          VARCHAR(8),
     PreReqID          VARCHAR(8),
     PRIMARY KEY (CourseID, PreReqID),
     FOREIGN KEY (CourseID) REFERENCES Course (CourseID) ON DELETE CASCADE,
     FOREIGN KEY (PreReqID) REFERENCES Course (CourseID)
    );
```

University Database Schema (3 of 4)

```
CREATE TABLE Teaches
(InstID          VARCHAR(5),
 CourseID        VARCHAR(8),
 SectionID       VARCHAR(8),
 Semester        ENUM('Fall','Winter','Spring','Summer'),
 StudyYear       YEAR,
 PRIMARY KEY(InstID, CourseID, SectionID, Semester, StudyYear),
 FOREIGN KEY(CourseID, SectionID, Semester, StudyYear)
   REFERENCES Section(CourseID, SectionID, Semester, StudyYear) ON DELETE CASCADE,
 FOREIGN KEY(InstID) REFERENCES Instructor(InstID) ON DELETE CASCADE
);

CREATE TABLE Takes
(StudID          VARCHAR(5),
 CourseID        VARCHAR(8),
 SectionID       VARCHAR(8),
 Semester        ENUM('Fall','Winter','Spring','Summer'),
 StudyYear       YEAR,
 Grade          VARCHAR(2),
 PRIMARY KEY(StudID, CourseID, SectionID, Semester, StudyYear),
 FOREIGN KEY(CourseID, SectionID, Semester, StudyYear)
   REFERENCES Section(CourseID, SectionID, Semester, StudyYear) ON DELETE CASCADE,
 FOREIGN KEY(StudID) REFERENCES Student(StudID) ON DELETE CASCADE
);
```

University Database Schema (4 of 4)

```
CREATE TABLE Section
    (CourseID          VARCHAR(8),
     SectionID         VARCHAR(8),
     Semester          ENUM('Fall','Winter','Spring','Summer'),
     StudyYear         YEAR,
     Building          VARCHAR(15),
     Room              VARCHAR(7),
     TimeSlotID        VARCHAR(4),
     PRIMARY KEY(CourseID, SectionID, Semester, StudyYear),
     FOREIGN KEY(CourseID) REFERENCES Course(CourseID) ON DELETE CASCADE,
     FOREIGN KEY(Building, Room) REFERENCES Classroom(Building, Room) ON DELETE SET NULL
    );
```

```
CREATE TABLE TimeSlot
    (TimeSlotID        VARCHAR(4),
     DayCode           ENUM('M','T','W','R','F','S','U'),
     StartTime         TIME,
     EndTime           TIME,
     PRIMARY KEY(TimeSlotID, DayCode, StartTime)
    );
```

```
CREATE TABLE Classroom
    (Building          VARCHAR(15),
     Room              VARCHAR(7),
     Capacity          DECIMAL(4,0),
     PRIMARY KEY(Building, Room)
    );
```

Database Instance (1 of 4)

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Student

StudID	StudName	Birth	DeptName	TotCredits
00128	Zhang	1992-04-18	Comp. Sci.	102
12345	Shankar	1995-12-06	Comp. Sci.	32
19991	Brandt	1993-05-24	History	80
23121	Chavez	1992-04-18	Finance	110
44553	Peltier	1995-10-18	Physics	56
45678	Levy	1995-08-01	Physics	46
54321	Williams	1995-02-28	Comp. Sci.	54
55739	Sanchez	1995-06-04	Music	38
70557	Snow	1995-11-22	Physics	0
76543	Brown	1994-03-05	Comp. Sci.	58
76653	Aoi	1993-09-18	Elec. Eng.	60
98765	Bourikas	1992-09-23	Elec. Eng.	98
98988	Tanaka	1992-06-02	Biology	120

Advisor

StudID	InstID
12345	10101
44553	22222
45678	22222
00128	45565
76543	45565
23121	76543
98988	76766
76653	98345
98765	98345

Department

DeptName	Building	Budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Database Instance (2 of 4)

Teaches

InstID	CourseID	SectionID	Semester	StudyYear
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
10101	CS-101	1	Fall	2009
45565	CS-101	1	Spring	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
10101	CS-315	1	Spring	2010
45565	CS-319	1	Spring	2010
83821	CS-319	2	Spring	2010
10101	CS-347	1	Fall	2009
98345	EE-181	1	Spring	2009
12121	FIN-201	1	Spring	2010
32343	HIS-351	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Takes

StudID	CourseID	SectionID	Semester	StudyYear	Grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	NULL

Database Instance (3 of 4)

Course

CourseID	Title	DeptName	Credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

PreReq

CourseID	PreReqID
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Database Instance (4 of 4)

Section

CourseID	SectionID	Semester	StudyYear	Building	Room	TimeSlotID
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

TimeSlot

TimeSlotID	DayCode	StartTime	EndTime
A	M	08:00:00	08:50:00
A	W	08:00:00	08:50:00
A	F	08:00:00	08:50:00
B	M	09:00:00	09:50:00
B	W	09:00:00	09:50:00
B	F	09:00:00	09:50:00
C	M	11:00:00	11:50:00
C	W	11:00:00	11:50:00
C	F	11:00:00	11:50:00
D	M	13:00:00	13:50:00
D	W	13:00:00	13:50:00
D	F	13:00:00	13:50:00
E	T	10:30:00	11:45:00
E	R	10:30:00	11:45:00
F	T	14:30:00	15:45:00
F	R	14:30:00	15:45:00
G	M	16:00:00	16:50:00
G	W	16:00:00	16:50:00
G	F	16:00:00	16:50:00
H	W	10:00:00	12:30:00

Classroom

Building	Room	Capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50